

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL ESCOLA
POLITÉCNICA
CURSO DE BACHARELADO EM ENGENHARIA DE SOFTWARE

TRABALHO 1 – SISTEMAS OPERACIONAIS
DUELO DE CONTEXTOS (PROCESSOS VS THREADS)

ALUNO: LUANA THUROW e SERGIO FILHO

PROFESSOR: FILIPO NOVO MÓR

PORTO ALEGRE, ABRIL DE 2026.

O Problema: O Contador Compartilhado

O programa que incremente um contador global até o valor de 1.000.000.000 (um bilhão), distribuindo o esforço entre N unidades de execução (onde N = 2, 4 e 8).

1. Implementação com Threads

Criar N threads que dividem o trabalho de incrementar a variável global.

Antes de entender o cenário com cada um dos mecanismos, precisamos entender que

```
counter_t contador = 0;
int N;

pthread_mutex_t lock;
```

O contador é global e compartilhado entre todas as threads, isso é, qualquer thread pode ler, modificar e escrever esse valor. Isso é exatamente o cenário clássico de condição de corrida (race condition).

T1. Sem nenhum mecanismo de sincronização:

```
void* incrementa_sem_mutex(void* arg) {
    for (counter_t i = 0; i < TOTAL / N; i++) {
        contador++;
    }
    return NULL;
}
```

O grande problema nesta opção é o contador, ele não é atômico. Ele precisa:

- Ler contador da memória
- Somar 1 localmente
- Escrever o valor de volta

Ao executar o T1, abaixo podemos observar:

```
luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$ gcc threads.c -o threads -pthread
luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$ ./threads 4 1
Contador final: 259403850
```

O resultado não é 1.000.000.000, além de o programa executar mais rapidamente. Sem mecanismo de sincronização, o contador global não atingiu o valor esperado de 1.000.000.000. Isso ocorre devido a condições de corrida, pois múltiplas threads acessam e modificam a variável compartilhada simultaneamente sem qualquer mecanismo de sincronização. A operação de incremento não é atômica, o que resulta na perda de incrementos quando duas ou mais threads executam o acesso concorrentemente.

T2. Com mecanismo de sincronização mutex:

```
void* incrementa_com_mutex(void* arg) {
    for (counter_t i = 0; i < TOTAL / N; i++) {
        pthread_mutex_lock(&lock);
        contador++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

Diferente do primeiro experimento, neste temos o ***pthread_mutex_lock(&lock)*** antes de incrementar, isso garante que thread por vez entra na seção crítica e nenhuma outra thread pode mexer em contador. Depois o ***pthread_mutex_unlock(&lock)***, que libera o acesso para outra thread.

```
• luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$ ./threads 4 2
  Contador final: 1000000000
○ luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$
```

O valor final sempre é exatamente 1.000.000.000 e o tempo de execução é maior. O crescimento de threads não escala bem. Isso porque as threads ficam disputando o mesmo mutex, muitas ficam bloqueadas esperando.

O uso de um mutex garante exclusão mútua durante o acesso ao contador global. Com isso, elimina-se a condição de corrida observada no experimento T1, e o valor final do contador atinge corretamente 1.000.000.000 em todas as execuções. No entanto, essa correção tem como custo um aumento significativo no tempo de execução, devido à contenção gerada pelo mutex, especialmente para valores maiores de N.

Comparação das Threads com e sem Sincronização:

	Sem Mutex	Com Mutex
Correção do resultado	Incorreto	Correta
Consistência dos dados	Não garantida	Garantida
Tempo de execução	Rápida	Lenta
Escalabilidade	Piora com mais threads	Limitada pelo mutex

2. Implementação com Processos

Em processos, a memória não é compartilhada por padrão. Para permitir o acesso concorrente ao contador global, utilizou-se memória compartilhada via System V IPC (shmget / shmat).

```
int N = atoi(argv[1]);
int modo = atoi(argv[2]);
```

```
int shmid = shmget(IPC_PRIVATE, sizeof(counter_t), IPC_CREAT | 0666);
counter_t *contador = (counter_t*) shmat(shmid, NULL, 0);

*contador = 0;
```

1. shmget cria um segmento de memória compartilhada
2. shmat conecta esse segmento ao espaço de endereçamento do processo
3. Após o fork():
4. Todos os processos filhos compartilham esse contador

Criação dos processos:

```
for (int i = 0; i < N; i++) {
    if (fork() == 0) {
        for (counter_t j = 0; j < TOTAL / N; j++) {
            if (modo == 1) {
                (*contador)++;
            } else {
                sem_wait(sem);
                (*contador)++;
                sem_post(sem);
            }
        }
        return 0;
    }
}
```

1. O processo pai cria N processos filhos
2. Cada filho:
3. Executa TOTAL / N incrementos
4. Sai com return 0
5. O pai:
6. Continua no loop
7. Depois chama wait() para todos

P1. Sem nenhum mecanismo de sincronização:

```
if (modo == 1) {
    (*contador)++;
}
```

Neste problema, é muito parecido com o das threads sem mutex só que pior:

Cada processo tem:

- Seu próprio cache
- Seu próprio escalonamento pelo SO

- Dois processos podem:
- Ler o mesmo valor
- Escrever o mesmo resultado
- Perder incrementos

O valor final não chega a 1 bilhão. O erro cresce com mais processos e mais núcleos do processador.

```
luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$ gcc processos.c -o processos -pthread
luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$ ./processos 4 1
Contador final: 257029496
```

Neste experimento, múltiplos processos acessam e modificam o contador na memória compartilhada sem qualquer mecanismo de sincronização. Como a operação de incremento não é atômica, ocorrem condições de corrida, resultando em perda de incrementos. Consequentemente, o valor final do contador é inferior ao valor esperado de 1.000.000.000, variando conforme o número de processos e o hardware utilizado.

P2. Utilização de Semáforos (sem_open) para proteger a memória.

```
sem = sem_open("/meu_semaforo", O_CREAT, 0644, 1);
```

- Cria um semáforo nomeado
- Valor inicial = 1
- Funciona como um mutex entre processos

Região crítica protegida

```
sem_wait(sem);
(*contador)++;
sem_post(sem);
```

- Apenas um processo por vez entra
- Incrementos não são perdidos
- Resultado correto

```
luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$ ./processos 4 2
Contador final: 1000000000
```

No experimento P2, utilizou-se um semáforo POSIX para garantir exclusão mútua no acesso à variável compartilhada. Com isso, eliminou-se a condição de corrida observada no experimento P1, e o valor final do contador atingiu corretamente 1.000.000.000 em todas as execuções. Entretanto, o uso do semáforo introduziu um overhead significativo, aumentando o tempo total de execução devido à contenção entre os processos.

Comparação Threads × Processos:

Modelo	Criação	Comunicação	Sincronização	Overhead
--------	---------	-------------	---------------	----------

Threads	Barata	Implícita (memória comum)	Mutex	Baixo
Processos	Cara (fork)	SHM explícita	Semáforo	Alto

3. Assinatura do Hardware: O resultado do comando lscpu (Linux).

```
luana.thurow@linux-ep:~/Documentos/T1-SISOP-O-Duelo-de-Contextos-Processos-vs.-Threads-/src$ lscpu
Arquitetura: x86_64
Modo(s) operacional da CPU: 32-bit, 64-bit
Tamanhos de endereço: 36 bits physical, 48 bits virtual
Ordem dos bytes: Little Endian
CPU(s): 4
Lista de CPU(s) on-line: 0-3
ID de fornecedor: GenuineIntel
Nome do modelo: Intel(R) Core(TM) i5-2400 CPU @ 3.10GHz
Família da CPU: 6
Modelo: 42
Thread(s) per núcleo: 1
Núcleo(s) por soquete: 4
Soquete(s): 1
Step: 7
CPU(s) scaling Mhz: 63%
CPU Mhz máx.: 3400,0000
CPU Mhz min.: 1600,0000
BogoMIPS: 6185,67
Opções: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse
sse2 ht tm pbe syscall nx rdtscp lm constant tsc arch perfmon pebs bts rep good nopl xtopology nonstop
_tsc cpuid aperfperf pni pclmulqdq dtes64 monitor ds cpl vmx smx est tm2 ssse3 cx16 xtpr pdcm pcid ss
e41 sse4_2 x2apic popcnt tsc deadline timer aes xsave avx lahf_lm epb pti ssbd ibrs ibpb stibp tpr_sh
adow flexpriority ept vpid xsaveopt dtherm ida arat pln pts vml md_clear flush_lid
Recursos de virtualização:
Virtualização: VT-x
Caches (soma de todos):
L1d: 128 KiB (4 instâncias)
L1i: 128 KiB (4 instâncias)
L2: 1 MiB (4 instâncias)
L3: 6 MiB (1 instância)
NUMA:
Número de NUMA: 1
CPU(s) de nó NUMA: 0-3
Vulnerabilidades:
Gather data sampling: Not affected
Indirect target selection: Not affected
Itlb multihit: KVM: Mitigation: Split huge pages
L1tf: Mitigation: PTE Inversion; VMX conditional cache flushes, SMT disabled
Mds: Mitigation: Clear CPU buffers; SMT disabled
Meltdown: Mitigation: PTI
Mmio stale data: Unknown: No mitigations
Reg file data sampling: Not affected
Retbleed: Not affected
Spec rstack overflow: Not affected
Spec store bypass: Mitigation: Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation: usercopy/swapgs barriers and user pointer sanitization
Spectre v2: Mitigation: Retpolines; IBRS conditional; IBRS_PV; STIBP disabled; RSB filling; PBRSE-eIBRS Not affect
ed; BHI Not affected
Srbds: Not affected
Taa: Not affected
Tsx async abort: Not affected
Vmscape: Mitigation: IBPB before exit to userspace
```

luana.thurow@linux-ep:~/Documentos/T1-SISOP-O-Duelo-de-Contextos-Processos-vs.-Threads-/src\$ lscpu

Arquitetura: x86_64
Modo(s) operacional da CPU: 32-bit, 64-bit
Tamanhos de endereço: 36 bits physical, 48 bits virtual
Ordem dos bytes: Little Endian
CPU(s): 4
Lista de CPU(s) on-line: 0-3
ID de fornecedor: GenuineIntel
Nome do modelo: Intel(R) Core(TM) i5-2400 CPU @ 3.10GHz
Família da CPU: 6
Modelo: 42
Thread(s) per núcleo: 1
Núcleo(s) por soquete: 4
Soquete(s): 1
Step: 7

CPU(s) scaling MHz: 63%
 CPU MHz máx.: 3400,0000
 CPU MHz mín.: 1600,0000
 BogoMIPS: 6185,67
 Opções: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr
 sse
 sse2 ht tm pbe syscall nx rdtscp lm constant_tsc arch_perfmon pebs bts rep_good noopl xtopology nonstop
 _tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 cx16 xtpr pdcm pcid
 ss
 e4_1 sse4_2 x2apic popcnt tsc_deadline_timer aes xsave avx lahf_lm epb pti ssbd ibrs ibpb stibp tpr_sh
 adow flexpriority ept vpid xsaveopt dtherm ida arat pln pts vnmi md_clear flush_l1d
 Recursos de virtualização:
 Virtualização: VT-x
 Caches (soma de todos):
 L1d: 128 KiB (4 instâncias)
 L1i: 128 KiB (4 instâncias)
 L2: 1 MiB (4 instâncias)
 L3: 6 MiB (1 instância)
 NUMA:
 Nó(s) de NUMA: 1
 CPU(s) de nó0 NUMA: 0-3
 Vulnerabilidades:
 Gather data sampling: Not affected
 Indirect target selection: Not affected
 Itlb multihit: KVM: Mitigation: Split huge pages
 L1tf: Mitigation; PTE Inversion; VMX conditional cache flushes, SMT disabled
 Mds: Mitigation; Clear CPU buffers; SMT disabled
 Meltdown: Mitigation; PTI
 Mmio stale data: Unknown: No mitigations
 Reg file data sampling: Not affected
 Retbleed: Not affected
 Spec rstack overflow: Not affected
 Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
 Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
 Spectre v2: Mitigation; Retpolines; IBPB conditional; IBRS_FW; STIBP disabled; RSB filling; PBRSE-eIBRS Not
 affect
 ed; BHI Not affected
 Srbds: Not affected
 Tsa: Not affected
 Tsx async abort: Not affected
 Vmscape: Mitigation; IBPB before exit to userspace

4. Tabela de Tempos: O tempo real de execução (usando o comando time) para cada cenário (T1, T2, P1, P2) variando o número de N (2, 4, 8).


```

luana.thurow@linux-ep:~/Documentos/T1-SISOP-O-Duelo-de-Contextos-Processos-vs.-Threads-/src$ time ./threads 2 1
time ./threads 2 2
time ./threads 4 1
time ./threads 4 2
time ./threads 8 1
time ./threads 8 2

time ./processos 2 1
time ./processos 2 2
time ./processos 4 1
time ./processos 4 2
time ./processos 8 1
time ./processos 8 2
Contador final: 396542585

real    0m13,464s
user    0m25,847s
sys     0m0,009s
Contador final: 1000000000

real    1m11,511s
user    1m51,248s
sys     0m29,818s
Contador final: 252536212

real    0m13,665s
user    0m48,468s
sys     0m0,040s
Contador final: 1000000000

real    1m9,704s
user    2m39,639s
sys     1m38,419s
Contador final: 137373732

real    0m13,674s
user    0m53,142s
sys     0m0,008s
Contador final: 1000000000

real    1m6,794s
user    2m22,598s
sys     1m56,495s
Contador final: 395069544

real    0m9,440s
user    0m18,834s
sys     0m0,001s
Contador final: 1000000000

real    1m39,784s
user    1m42,121s
sys     1m31,221s
Contador final: 336300976

real    0m5,324s
user    0m17,957s
sys     0m0,013s
Contador final: 1000000000

real    3m54,073s
user    6m15,826s
sys     8m2,651s
Contador final: 180260407

real    0m4,363s
user    0m16,951s
sys     0m0,008s
Contador final: 1000000000

real    2m57,713s
user    5m24,207s
sys     6m14,228s
luana.thurow@linux-ep:~/Documentos/T1-SISOP-O-Duelo-de-Contextos-Processos-vs.-Threads-/src$

```

luana.thurow@linux-ep:~/Documentos/T1-SISOP-O-Duelo-de-Contextos-Processos-vs.-Threads-/src\$

```

time ./threads 2 1
time ./threads 2 2
time ./threads 4 1
time ./threads 4 2
time ./threads 8 1
time ./threads 8 2

```

```

time ./processos 2 1

```

```
time ./processos 2 2
time ./processos 4 1
time ./processos 4 2
time ./processos 8 1
time ./processos 8 2
```

```
Contador final: 396542585
real  0m13,464s
user  0m25,847s
sys   0m0,009s
```

```
Contador final: 1000000000
real  1m11,511s
user  1m51,248s
sys   0m29,818s
```

```
Contador final: 252536212
real  0m13,665s
user  0m48,468s
sys   0m0,040s
```

```
Contador final: 1000000000
real  1m9,704s
user  2m39,639s
sys   1m38,419s
```

```
Contador final: 137373732
real  0m13,674s
user  0m53,142s
sys   0m0,008s
```

```
Contador final: 1000000000
real  1m6,794s
user  2m22,598s
sys   1m56,495s
```

```
Contador final: 395069544
real  0m9,440s
user  0m18,834s
sys   0m0,001s
```

```
Contador final: 1000000000
real  1m39,784s
user  1m42,121s
sys   1m31,221s
```

```
Contador final: 336300976
real  0m5,324s
```

user 0m17,957s
sys 0m0,013s

Contador final: 1000000000

real 3m54,073s
user 6m15,826s
sys 8m2,651s

Contador final: 180260407

real 0m4,363s
user 0m16,951s
sys 0m0,008s

Contador final: 1000000000

real 2m57,713s
user 5m24,207s
sys 6m14,228s

Cenário	N = 2	N = 4	N = 8
T1 – Threads sem sincronização	13,464 s	13,665 s	13,674 s
T2 – Threads com mutex	1m 11,511s	1m 09,704s	1m 06,794s
P1 – Processos sem sincronização	9,440 s	5,324 s	4,363 s
P2 – Processos com semáforo	1m 39,784s	3m 54,073s	2m 57,713s

Os experimentos sem sincronização (T1 e P1) apresentam tempos significativamente menores, porém com resultados incorretos.

Os experimentos com sincronização (T2 e P2) garantem correção, mas apresentam alto overhead, especialmente no caso de processos com semáforo (P2).

O aumento de N melhora o tempo em alguns cenários sem sincronização (especialmente P1), mas piora a escalabilidade nos cenários sincronizados devido à contenção.

5. Análise de Corrupção: No experimento T1 e P1, qual foi o valor final do contador? Explique por que ele não chegou em 1 bilhão e como a variação do hardware do grupo influenciou o erro.

O experimento foi executado em um sistema com as seguintes características principais:

- Processador: Intel Core i5-2400 @ 3.10 GHz
- Núcleos físicos: 4
- Threads por núcleo: 1 (não há Hyper-Threading)

- Total de CPUs lógicas: 4
- Arquitetura: x86_64
- Cache:
 - L1: 128 KB (4 instâncias)
 - L2: 1 MB (4 instâncias)
 - L3: 6 MB (compartilhado)

T1 Valor final do contador: 259403850

```
luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$ gcc threads.c -o threads -pthread
luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$ ./threads 4 1
Contador final: 259403850
```

P1 Valor final do contador: 257029496

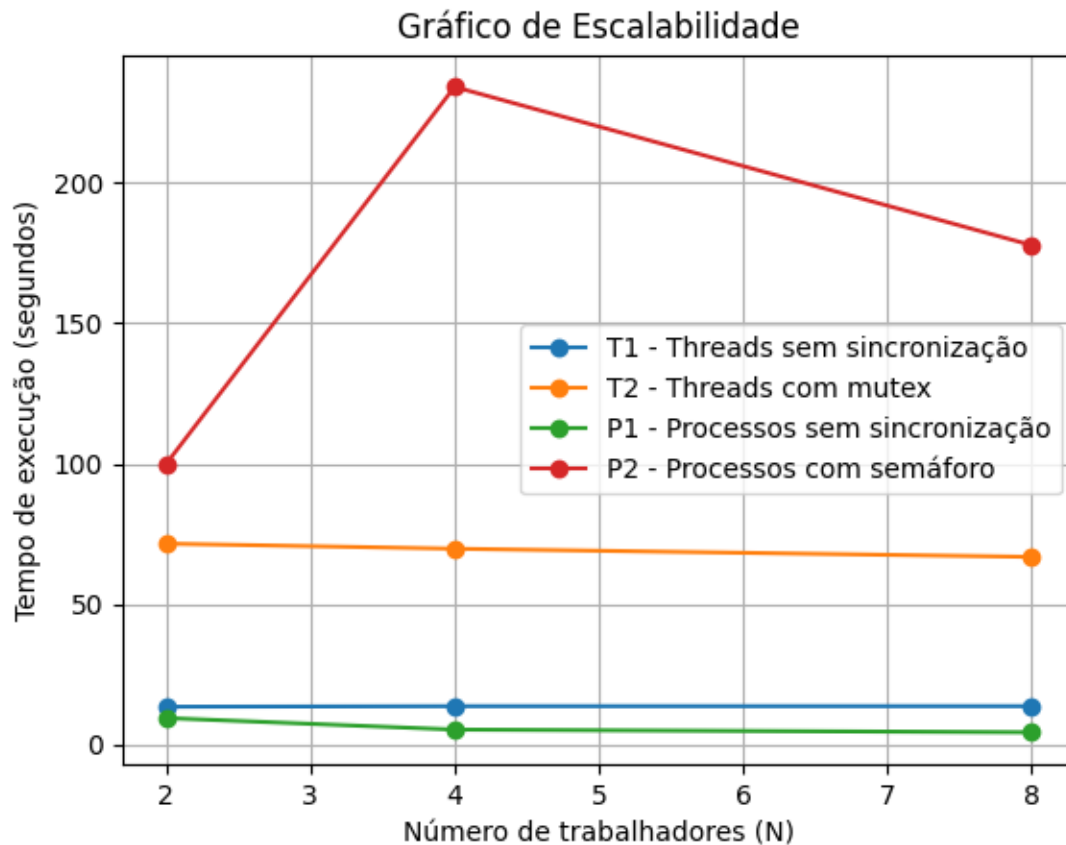
```
luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$ gcc processos.c -o processos -pthread
luana.thurow@linux-ep:~/Documentos/T1-SISOP-0-Duelo-de-Contextos-Processos-vs.-Threads-/src$ ./processos 4 1
Contador final: 257029496
```

Em ambos os casos, o contador não atingiu o valor esperado de 1.000.000.000 devido à ocorrência de condições de corrida.

A operação de incremento (contador++) não é atômica, pois envolve leitura, modificação e escrita na memória. Quando múltiplas threads ou processos executam essa operação simultaneamente, incrementos podem ser perdidos. Por exemplo, duas unidades de execução podem ler o mesmo valor do contador e escrever o mesmo resultado incrementado, causando a perda de um incremento.

A variação do hardware influenciou diretamente esse erro. O sistema possui 4 núcleos físicos, permitindo execução paralela real, o que aumenta a concorrência no acesso ao contador compartilhado. Além disso, a hierarquia de cache contribui para inconsistências temporárias entre os núcleos. Os valores próximos, porém, diferentes, entre T1 e P1 reforçam o caráter não determinístico do problema, que depende do escalonamento do sistema operacional e das características do hardware

6. Gráfico de Escalabilidade: Um gráfico (pode ser gerado em Python/Excel) mostrando o tempo de execução vs. número de trabalhadores (N).



O gráfico de escalabilidade evidencia que os experimentos sem sincronização (T1 e P1) apresentam menor tempo de execução, porém resultados incorretos. Já os experimentos com sincronização (T2 e P2) garantem consistência dos dados, mas apresentam maior overhead e pior escalabilidade, especialmente no caso de processos com semáforos.

Referências Bibliográficas

THUROW, Luana. T1-SISOP – O Duelo de Contextos: Processos vs Threads. Repositório GitHub, 2026. Disponível em: <<https://github.com/luanathurow/T1-SISOP-O-Duelo-de-Contextos-Processos-vs-Threads-.git>>. Acesso em: 22 abr. 2026.

BAELDUNG. Process vs Thread: What's the Difference? Disponível em: <<https://www.baeldung.com/cs/process-vs-thread>>. Acesso em: 22 abr. 2026.

MICROSOFT BING. Resultados de busca: gráfico python. Disponível em: <<https://www.bing.com/search?q=grafico+python>>. Acesso em: 22 abr. 2026.